TraceGap: Measuring Whether LLM Agent Frameworks Record the Tool Calls Their Agents Make

Anonymous authors
Paper under double-blind review

Abstract

Debugging, auditing, evaluation and training of LLM agents all consume the record that agent frameworks keep of tool calls, yet benchmarks for tool use evaluate whether *models* call tools correctly, and to our knowledge none evaluates whether *frameworks* record those calls correctly. We present TraceGap, a benchmark and dataset that drive agent frameworks with scripted models and compare each framework's own trace surface (callbacks, tracing spans, OpenTelemetry and OpenInference spans, events, message history) with ground truth that does not depend on the framework: the calls the model emitted and an execution ledger written by the tools themselves. Each case is scored for completeness and fidelity. Across nine widely used Python frameworks and fifteen trace surfaces, 586 of 660 cases in 22 hand-written fault scenarios were complete (88.8%), and 3,746 of 6,000 in a seeded generated suite (62.4%). Rankings use a Stable tier, defined by a structural rule that contained every verdict that varied across repeated runs; 74.6% of its 3,090 cases were complete, against 56.8% in the race-prone Stress tier. Failures concentrate in rejected calls, cancellation, failures inside parallel batches and nested agents, and a recall-only metric, the obvious first choice, would miss 73% of them. A common pattern, span handling that does not see Python's cancellation exception, makes cancelled calls look successful in three frameworks, and in a fourth once another defect is fixed; TraceGap found defects in five frameworks, for which we provide fixes. A deliberately correct reference loop scores perfectly and eleven injected recording defects are each detected. An ablation shows that both ground-truth sources are needed: either alone misses or misreports a large share of failures. The released corpus of about 11,500 labeled cases defines four learning tasks; learned detectors of incomplete traces work well on the held-out test frameworks but vary widely across frameworks, classifying and reconstructing what was lost is harder, random splits substantially overestimate generalization compared with leave-one-framework-out evaluation, and features that identify the trace surface hurt it.

1 Introduction

LLM agents act through tools, and the record of those tool calls is what the rest of the stack consumes. Engineers debug from it, auditors review it, observability back ends aggregate it, trace-based evaluators and LLM judges score it (Deshpande et al., 2025; Zhang et al., 2025), and trajectories collected from it become training data. That record is usually produced neither by the model nor by the application but by the *agent framework*: LangGraph, the OpenAI Agents SDK, Pydantic AI, AutoGen and others expose callbacks, tracing spans, OpenTelemetry (OpenTelemetry Authors, 2026) or OpenInference (Arize AI, 2026) spans, event streams and message histories. If a framework drops a call, reports a cancelled call as successful, or loses the calls of a run that failed, every consumer inherits the error without knowing it.

Benchmarks for tool use evaluate the model: whether it selects the right tool and arguments (Patil et al., 2023; Qin et al., 2024), or completes tasks in interactive environments (Yao et al., 2024; Liu et al., 2024). Work on agent traces analyses them, to localize agent errors (Deshpande et al., 2025; Cemri et al., 2025) or reconstruct provenance (Wang et al., 2026), and takes the recorded trace as correct. Yet an empirical study of LangChain and CrewAI found telemetry malfunctions among the root causes of framework bugs (Zhu

et al., 2026). The layer underneath has not been measured: *given that a call happened, does the framework's trace say so, accurately?*

Measuring this is harder than it looks. Ground truth cannot come from the framework under test, real models make runs irreproducible, and the interesting conditions (a tool raising inside a batch of parallel calls, cancellation mid-call, a call to a tool that does not exist) are rare in natural traffic. TraceGap addresses these with scripted models, tools that write their own execution ledger, and fault scenarios that make rare conditions routine (Figure 1).

Research questions. We organize the study around six questions:

RQ1 How completely and faithfully do current agent frameworks record tool execution? (§5)

RQ2 Under which execution conditions do surfaces fall short of the contract, and why? (§5.1, §5.3)

RQ3 Are recording verdicts stable across repeated executions, and is the benchmark's design necessary? (§4)

RQ4 How has recording correctness changed across framework releases? (§7)

RQ5 Can trace defects be detected from what a pipeline observes, without the execution ledger? (§8)

RQ6 Do learned trace-defect detectors generalize to agent frameworks they have never seen? (§8)

Contributions.

- **A measurement method** with framework-independent ground truth, formal completeness and fidelity criteria, a calibration case and a guard that never lets a case that did not run count as a pass (§3).
- **A benchmark** of 22 hand-written fault scenarios and a seeded generator, run on nine frameworks and fifteen trace surfaces in two execution modes, with **Stable and Stress tiers** that separate deterministic verdicts from race-prone ones by a rule fixed before it was tested on fresh runs (§3, §4).
- **Validation of the benchmark**: a reference loop that scores perfectly, mutation testing with eleven injected defects on both suites, an ablation showing that both ground-truth sources are needed, repeated runs, bootstrap confidence intervals and ranking agreement across independently generated suites (§4).
- **Findings**: where and why framework traces fall short of the contract, a metric ablation showing why the criteria matter, a cross-framework root cause, defects in five frameworks with fixes, and a history across 65 releases (§5–§7).
- **A dataset and four learning tasks** over held-out frameworks, with rule-based, feature-based, bag-of-n-gram and transformer baselines, and a comparison of random, held-out-framework and leave-one-framework-out evaluation showing that random splits substantially overestimate generalization to unseen frameworks (§8).

2 Related work

We searched systematically for prior work in October 2026, with twelve queries including "agent observability benchmark", "LLM tracing benchmark", "OpenTelemetry LLM benchmark", "agent provenance benchmark", "tool call tracing evaluation", "LLM trace completeness" and "agent execution provenance dataset"; the queries, inclusion criteria and per-candidate assessment are released with the benchmark. We found no benchmark or dataset that compares what agent frameworks record about tool calls with what executed, using ground truth independent of the framework.

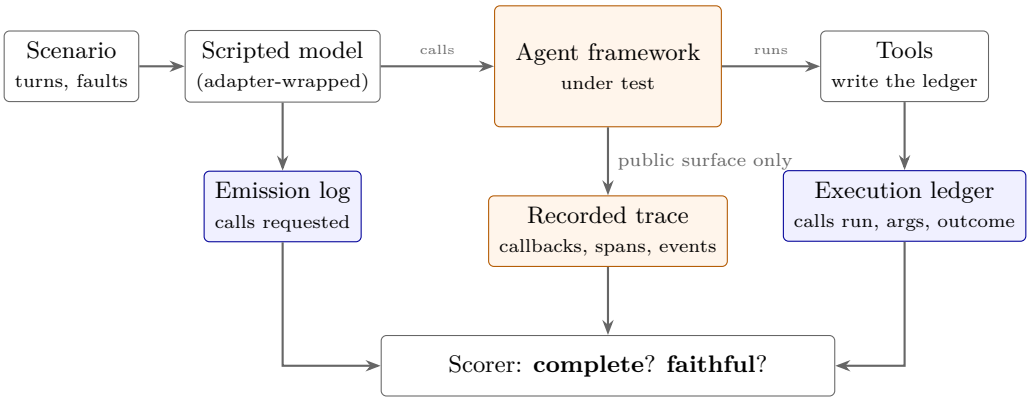


Figure 1: TraceGap. Ground truth (blue) is written by the scripted model and by the tools themselves; the framework's own record (orange) is read only through its public observability surface. The scorer compares the three logs.

Tool-use and agent benchmarks score the model: tool selection and arguments (Patil et al., 2023; Qin et al., 2024) or task success in environments (Yao et al., 2024; Liu et al., 2024). They take the framework's execution and its record for granted.

Work that consumes traces. TRAIL annotates 148 OpenInference traces with agent reasoning, execution and planning errors (Deshpande et al., 2025); Who&When and MAST attribute failures in multi-agent systems from their logs (Zhang et al., 2025; Cemri et al., 2025). These label errors *of the agent* and treat the trace as given. A survey of evidence tracing and execution provenance describes the area as fragmented and lists unified trace schemas as an open problem (Wang et al., 2026). AgentTrace proposes a structured logging layer (AlSayyad et al., 2026), and fault-injection work uses OpenTelemetry to study how injected delays propagate through multi-agent systems (Seyedghorban et al., 2026). None tests whether the record itself is correct.

Datasets of agent traces. AgentSec provides 30 synthetic scenarios with tool calls and W3C PROV graphs for auditing reasoning consistency (Hmimou et al., 2026). Its traces are generated from specifications rather than recorded by frameworks, so recording defects cannot arise in it.

Evidence that recording fails. Zhu et al. classify 998 bug reports from LangChain and CrewAI and attribute 32 to "telemetry malfunction" (Zhu et al., 2026). Qin et al. show that agents can delete or alter their own traces when incentivized (Qin et al., 2026), an adversarial counterpart to the honest recording errors studied here. Both argue for records that do not depend on the component being audited, which is how TraceGap obtains ground truth. The OpenTelemetry GenAI conventions and OpenInference define span formats for tool execution (OpenTelemetry Authors, 2026; Arize AI, 2026) but come with no correctness suite for agent frameworks; TraceGap reads both.

Methodology. Our benchmark validation borrows mutation testing from software engineering (DeMillo et al., 1978): a measurement instrument is trusted when it detects deliberately injected faults. Uncertainty is quantified with the bootstrap (Efron & Tibshirani, 1993) and ranking agreement with Kendall's τ (Kendall, 1938). The dataset is documented with a datasheet (Gebru et al., 2021).

3 The TraceGap benchmark

3.1 Ground truth

A *case* is a scenario run on one trace surface in one execution mode. It produces two logs that do not depend on the framework:

- the **emission log** E: every tool call the scripted model returned, with its call id, name, arguments, run and nesting level;
- the **execution ledger** L: every tool call that started, written from inside the tool body, with its real arguments and a final status in $\{\mathsf{ok}, \mathsf{error}, \mathsf{cancelled}, \mathsf{running}\}$, output and error message.

Every tool takes a unique `tag` argument, so records are matched to ground truth without trusting framework-assigned identifiers; surfaces that do not record arguments are matched by the model's call id. The framework's **recorded trace** R is read only through its public observability interface and normalized to records $(\text{name}, \text{args}, \text{status}, \text{output}, \text{error}, \text{call id}, \text{group})$, where *group* is the run identity the surface reports (a trace id, a root run id, a result object). Recorded statuses are ok, error, incomplete or unknown.

3.2 Scripted models and adapters

An adapter wraps a shared script cursor in the framework's own model interface, using the framework's test model where one exists, builds an agent with the scenario's tools, runs it, and converts what the surface reported into records. Adapters never rebuild events from the ledger and never patch framework internals. Each surface declares which fields it captures, and is scored only on those. Two rules keep comparisons fair. When a framework asks the model for a tool-less final answer (for example after a turn limit), the scripted model answers in text without consuming the script, as a real model offered no tools would. A case in which no call was emitted is a harness error, never a pass; this guard caught several adapter and compatibility bugs during development. The single-call scenario S01 serves as a calibration case: a surface that fails it is not being read correctly.

3.3 Metrics

Let $L_{\text{run}} \subseteq L$ be the calls that started, $E \setminus L$ the calls that were requested but never started (rejected by the framework, or never reached), and $m(\cdot)$ the matching from ledger and emission entries to records. A case is **complete** when all of the following hold:

(C1) *recall*: every $\ell \in L_{\text{run}}$ has a record;

(C2) *honest status*: no record of a call whose ledger status is error, cancelled or running, or that never ran, says ok; a record of a successful call is not error, and incomplete is accepted only for a call that did not finish;

(C3) *attempt visibility*: every $e \in E \setminus L$ is visible as a record;

(C4) *no phantoms or duplicates*: every record matches an emitted call, and no call is recorded more often than it executed;

(C5) *isolation*: no group mixes records of two runs, and no run is split across unrelated groups.

A case is **faithful** when every captured argument, output and error message matches the ledger exactly (outputs may be serialized; truncation is a failure). We additionally report, without scoring it, any tool still running after its run ended.

An observability contract, not a verdict on design. Together, completeness and fidelity form TraceGap's *observability contract*: what a consumer of a tool-level trace needs in order to reconstruct what happened. A surface that does not meet it is *incomplete under the contract*, which is not the same as being wrong. Some frameworks deliberately keep rejected calls out of their tool layer, or redact errors for privacy; the contract reports the consequence for consumers without judging the choice. We call behavior a *defect* only where it contradicts the surface's own semantics or the project treats it as a bug, as when a call that was cancelled is reported as successful (§6). Throughout, "complete" and "incomplete" are meant in this sense.

3.4 Scenarios

The **core suite** has 22 hand-written scenarios in seven categories (Appendix A): basic accounting (single, sequential, five parallel calls, a 25-turn session); tool failures (one raising call, failure then retry, one failure in a parallel batch); rejected calls (unknown tool, wrong argument type, missing argument); duplicate provider call ids; payload fidelity (unicode, a 200,000-character output, a dict, `None`, a synchronous tool); interruption (cancellation during a tool, cancellation after tools, a provider error, a turn limit); and structure (two concurrent runs, an agent nested as a tool). Cancellation uses `asyncio.Task.cancel()`, the mechanism behind timeouts and dropped requests.

The **generator** composes the same primitives into random but reproducible scenarios: one to five turns of one to four calls drawn from ten weighted call kinds (two kinds of successful call, raising, unknown tool, wrong type, missing argument, large output, structured output, `None` output, synchronous), one of five endings (final answer, cancellation in a tool, cancellation while the model thinks, provider error, turn limit), and with some probability two concurrent runs or a nested agent. Timing-dependent endings keep generous margins. A suite is identified by (n, seed); the released suite is n=200, seed 1, with 1,744 calls. Every scenario runs in *invoke* and *stream* mode.

3.5 Stable and Stress tiers

A ranking that changes when the benchmark is rerun is of little use. We therefore split scenarios by a structural rule that is known before a scenario runs: a scenario is **Stress** when any model turn emits two or more calls and at least one of them raises; otherwise it is **Stable**. How a framework handles the siblings of a failing call (whether they started, finished or were recorded before the run aborted) can depend on scheduling. One core scenario (S07) and 118 of the 200 generated scenarios are Stress. Each surface has 206 Stable and 238 Stress cases across both suites and modes. The leaderboard ranks surfaces on Stable cases only, CI usage defaults to them, and Stress results are reported separately. Section 4 tests the rule on runs made after it was fixed.

4 Validating the benchmark

Reference loop. A deliberately correct agent loop of about 100 lines (record the attempt before validating or executing; close the record on every exit path; never drop a requested call; tag every record with its run) is complete and faithful on all 44 core and all 400 generated cases. A perfect score is achievable, so failures reflect the surface rather than an impossible standard.

Mutation testing. We injected eleven recording defects into the reference loop one at a time and ran both suites (Table 1). Every defect is detected, by the scenarios designed for it and on the intended axis: truncation and lost error detail are fidelity failures but not completeness failures, while dropped or misreported calls are completeness failures. On the generated suite every defect is detected in at least 28 of 400 cases, so the generator exercises each check without hand-written targeting. These checks, together with a negative control for every scorer check, run in the test suite.

Why two sources of ground truth. To test whether the emission log and the execution ledger are both necessary, we re-scored every current-release case with only one of them (Table 2). With the emission log alone, a scorer knows what was requested but not what ran or how it ended: it catches 1,866 of the 2,328 incomplete cases (80.2%), misses every case whose only failure is a dishonest status, and cannot tell a call that ran and was dropped from one that never ran, which is ambiguous in 1,866 cases. With the ledger alone, a scorer knows what ran but not what was requested, and faces a dilemma over records of calls it never saw. Treated strictly, as phantoms, they cause false alarms on 1,696 of the 4,332 complete cases (39.2%), because a surface that correctly records a rejected call appears to report a call that never happened; this scorer catches 1,372 incomplete cases (58.9%). Treated leniently, accepted unless they claim success, they cause only 2 false alarms, but the scorer then catches just 1,098 (47.2%) and misses every case whose only failure is an invisible attempt. Only the combination of both sources gives verdicts that are both sensitive

Injected defect	What the loop does wrong	Core scenarios detecting	Axis	Generated
`drop_rejected`	rejected calls (unknown tool, bad arguments) are not recorded	S08–S10	C	252/400
`cancelled_as_ok`	a cancelled call is closed as a success	S17	C	50/400
`drop_unfinished`	calls that never finish are removed from the record	S17	C	50/400
`failed_as_ok`	a call that raised is recorded as a success	S05–S07, S09–S10	C+F	312/400
`shared_group`	every run is recorded under the same trace id	S21	C	28/400
`split_group`	every call gets its own trace id	S02–S04, S06–S07, S11, S17–S22	C	378/400
`truncate_output`	outputs are cut to 1,000 characters	S13	F	102/400
`drop_error_detail`	error messages are not recorded	S05–S07	F	234/400
`drop_nested`	calls made inside a nested agent are not recorded	S22	C	30/400
`double_record`	parallel calls are recorded twice	S03, S07, S11	C	374/400
`alter_args`	arguments are re-serialized lossily (non-ASCII replaced)	S12	F	294/400

Table 1: Mutation testing. Each defect is injected alone into the reference loop. Axis: C completeness, F fidelity. Generated: cases (of 400) where the defect is detected. Without defects the loop is complete and faithful on every case of both suites.

Ground truth used	Incomplete cases detected	False alarms	What is lost
Emissions + ledger (TraceGap)	2,328/2,328	0	–
Emission log only	1,866/2,328 (80.2%)	2	status honesty: misses all 462 cases whose only failure is a wrong status; cannot say whether a missing call ran
Ledger only, strict	1,372/2,328 (58.9%)	1,696	attempt visibility: misses 956 cases; correctly recorded rejected calls look like phantoms
Ledger only, lenient	1,098/2,328 (47.2%)	2	accepts unmatched records unless they claim success: few false alarms, misses 1,230 cases

Table 2: Ground-truth ablation on the 6,660 current-release cases (2,328 incomplete according to the full scorer, which defines the reference, so its row is correct by construction). False alarms: complete cases a single-source scorer would flag.

and specific. (The handful of false alarms for the single-source scorers come from duplicate call ids in one surface, where records cannot be matched to calls without both logs.)

Stability (RQ3). All 660 core verdicts were identical across four full runs (the last two are released). Three full runs of the generated suite, all released, agreed on 5,852 of 6,000 verdicts (97.5%). The Stress rule was fixed after the first two generated runs, in which 104 verdicts differed, all inside it. The third run, made afterwards, added 44 differing verdicts, again all inside it: 148 in total (119 for Pydantic AI's history, 18 for its OpenTelemetry spans, 11 for LangGraph). In every one, a call in a parallel batch raises and the framework's handling of the sibling calls is a race: in Pydantic AI, whether a sibling's result is recorded before the run aborts; in LangGraph, whether a sibling had started. Stable verdicts (3,090 cases) were identical in every run.

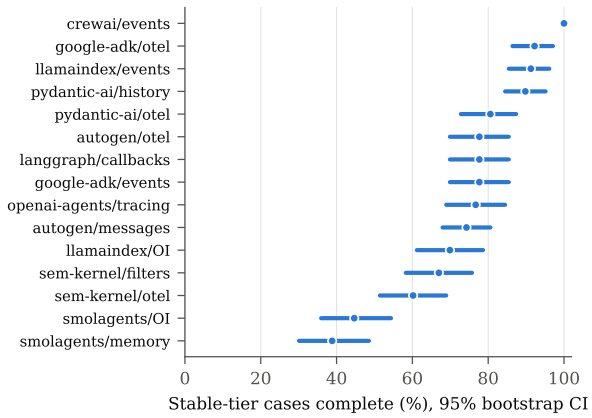


Figure 2: Stable-tier completeness per surface with 95% bootstrap confidence intervals (scenario resampling).

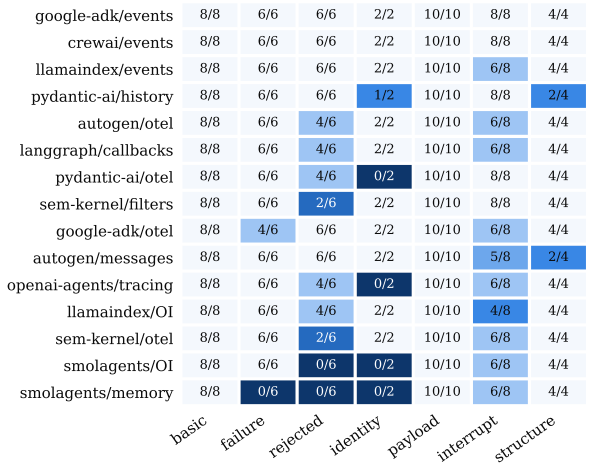


Figure 3: Complete cases per scenario category in the core suite (both modes). Darker cells mean more incomplete cases.

Uncertainty. Scenarios are a sample from a space of possible scenarios, so we report 95% bootstrap confidence intervals for each surface's Stable completeness, resampling the 103 Stable scenarios with both modes kept together (2,000 replicates; Figure 2). Intervals are 11–18 points wide for every surface except CrewAI's events, which is complete in every case. Of the 14 adjacent pairs in the ranking, 5 are separated in at least 95% of replicates: `crewai/events` > `google-adk/otel`, `pydantic-ai/history` > `pydantic-ai/otel`, `semantic-kernel/filters` > `semantic-kernel/otel`, `semantic-kernel/otel` > `smolagents/openinference`, `smolagents/openinference` > `smolagents/memory`. The leaderboard should therefore be read as ordered groups rather than a strict order; Table 3 prints the intervals beside each rank for this reason.

Agreement across suites. Rankings on the core and generated Stable cases agree with Kendall $\tau = 0.71$, although the suites share no scenario. We also generated 2 further suites with different seeds (n=100; seeds 2, 3) and ran them on all surfaces. Stable-tier rankings on the 3 independently generated suites agree with Kendall τ between 0.77 and 0.80 (mean 0.78; Appendix C), so the ranking is not an artifact of one sample of scenarios.

5 Results (RQ1, RQ2)

We evaluate LangGraph 1.2.12, the OpenAI Agents SDK 0.23.1, Pydantic AI 2.54.0, AutoGen 0.7.5, smolagents 1.26.0, LlamaIndex 0.14.25, Semantic Kernel 1.44.1, Google ADK 2.11.0 and CrewAI 1.15.23 on Python 3.13, reading fifteen surfaces (Appendix B). Table 3 gives the leaderboard. Across both suites, 2,304 of 3,090 Stable cases (74.6%) and 2,028 of 3,570 Stress cases (56.8%) were complete; on the core suite alone 586 of 660 (88.8%), and on the generated suite 3,746 of 6,000 (62.4%). One surface, CrewAI's event bus, is complete and faithful on every case. This should not be read as a general claim about CrewAI's observability stack: TraceGap evaluates the selected surface on the fields it exposes, and CrewAI's events are attributed to runs by task id, information the application has, which makes the isolation criterion easier for this surface than for those that must report run identity themselves. A full run of both suites on all surfaces takes about 47 minutes of wall-clock time without API keys or network (Appendix E).

5.1 Where traces fall short

Figure 3 breaks the core suite down by category. Basic accounting and payload scenarios are complete on every surface; rejected calls (30 of the 74 incomplete core cases) and interruption (23) account for most

		Stable tier (ranked)					Stress	Complete per suite		
#	Surface	Complete % [95% CI]	Faithful %	Fail.	Canc.	Rej.	Nest.	Complete %	Core	Gen.	Fail→ok
1	`crewai/events`	100.0 [100, 100]	100.0	4/4	4/4	6/6	2/2	100.0	44/44	400/400	0
2	`google-adk/otel`	92.2 [86, 97]	100.0	4/4	2/4	6/6	2/2	99.2	40/44	386/400	20
3	`llamaindex/events`	91.3 [85, 96]	100.0	4/4	4/4	6/6	2/2	82.4	42/44	342/400	0
4	`pydantic-ai/history`	89.8 [84, 95]	90.3	4/4	4/4	6/6	0/2	48.3	41/44	259/400	0
5	`pydantic-ai/otel`	80.6 [73, 87]	100.0	4/4	4/4	4/6	2/2	77.3	40/44	310/400	0
6	`autogen/otel`	77.7 [70, 85]	100.0	4/4	2/4	4/6	2/2	62.2	40/44	268/400	0
7	`langgraph/callbacks`	77.7 [70, 85]	100.0	4/4	4/4	4/6	2/2	59.7	40/44	262/400	0
8	`google-adk/events`	77.7 [70, 85]	66.0	4/4	4/4	6/6	2/2	14.3	44/44	150/400	0
9	`openai-agents/tracing`	76.7 [69, 84]	90.3	4/4	2/4	4/6	2/2	62.2	38/44	268/400	36
10	`autogen/messages`	74.3 [68, 81]	100.0	4/4	2/4	6/6	0/2	74.4	39/44	291/400	0
11	`llamaindex/openinference`	69.9 [61, 79]	100.0	4/4	2/4	4/6	2/2	47.9	38/44	220/400	0
12	`semantic-kernel/filters`	67.0 [58, 76]	100.0	4/4	4/4	2/6	2/2	48.7	40/44	214/400	0
13	`semantic-kernel/otel`	60.2 [51, 69]	100.0	4/4	2/4	2/6	2/2	40.3	38/44	182/400	52
14	`smolagents/openinference`	44.7 [36, 54]	100.0	4/4	2/4	0/6	2/2	29.4	34/44	128/400	0
15	`smolagents/memory`	38.8 [30, 49]	100.0	0/4	2/4	0/6	2/2	5.9	28/44	66/400	0

Table 3: Leaderboard. Stable tier: 206 cases per surface from both suites and modes. Capability columns count complete Stable cases among the core scenarios for tool failures (S05–S06), cancellation (S17–S18), rejected calls (S08–S10) and nested agents (S22). Fail→ok: failed or cancelled calls recorded as successful (both suites).

Condition (generated suite)	Scenarios	% complete with	% complete without	Logit coef.
parallel calls in a turn	192	61.2	91.2	−1.23
a tool raises	126	58.0	70.0	−0.50
a tool raises inside a parallel batch (Stress)	118	56.6	70.9	–
a call is rejected	133	53.4	80.3	−1.50
run is cancelled	52	55.4	64.9	−0.70
model call fails	27	62.3	62.4	−0.10
turn limit reached	29	63.4	62.3	−0.33
two concurrent runs	14	61.0	62.5	−0.05
nested agent	15	56.0	63.0	−0.37
synchronous tool	103	59.7	65.3	+0.16
large output	53	65.3	61.4	+0.37

Table 4: Completeness on the generated suite (all fifteen surfaces, 6,000 cases) by condition. The logistic regression includes all conditions except the Stress indicator (which is their conjunction), the log number of calls and surface fixed effects.

failures, followed by duplicate call ids (9), tool failures (8) and structure (4). To separate the contributions of conditions that co-occur in generated scenarios, Table 4 compares completeness with and without each condition and fits a logistic regression of completeness on all conditions, the log number of calls and surface fixed effects. Rejected calls ($\beta = -1.50$), parallel calls (-1.23), cancellation (-0.70) and a raising tool (-0.50) have the largest effects, followed by nested agents (-0.37); once these are accounted for, provider errors, turn limits and concurrent runs have little effect on average, though they matter for specific surfaces (§5.3). Completeness also falls with scenario size (log-calls coefficient -0.32): pooled over surfaces, from 83% for scenarios with one to three calls to 51% for those with sixteen or more (Figure 4). The decline is surface-specific: Google ADK's spans stay near 100%, LangGraph's callbacks stay flat, and smolagents' memory falls from 65% to 7%, because a single failing call removes every call of its step.

5.2 Why the criteria matter: a metric ablation

The obvious first metric for a trace is recall: does every executed call appear? Of the 2,328 incomplete cases across both suites, 1,710 (73%) have no missing executed call and would pass a recall-only check. Adding honest status still passes 1,230 of them (53%). The dominant failure is invisible attempts (1,617 cases, 1,230 with no other failure): the model asked for a call, the framework rejected it before execution, and

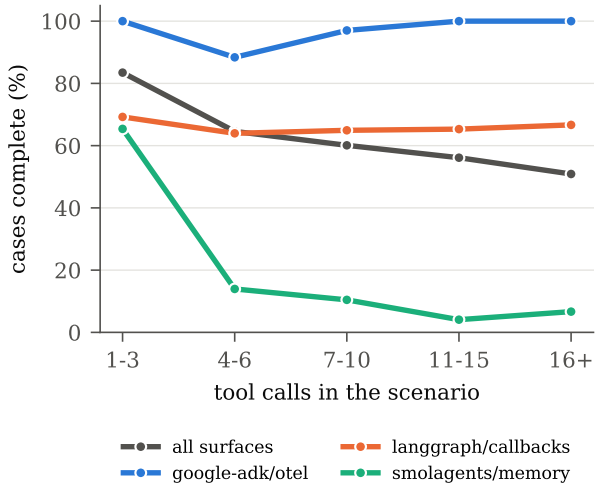


Figure 4: Completeness against the number of tool calls in a generated scenario, pooled over all surfaces and for three surfaces with different behavior.

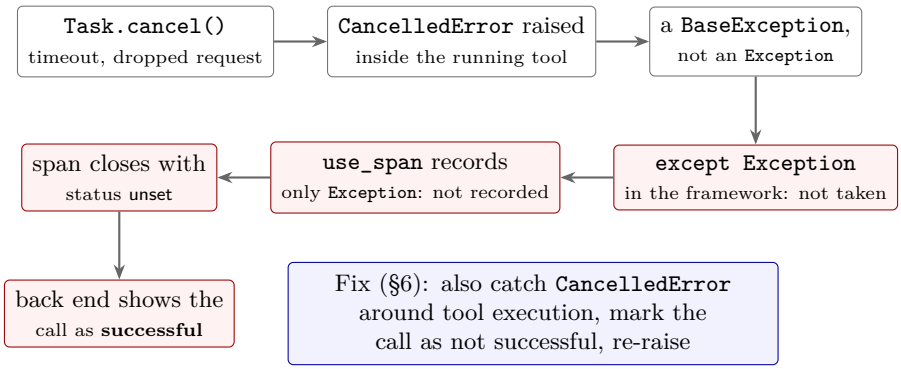


Figure 5: Why cancelled tool calls look successful. Semantic Kernel and Google ADK follow this path through `use_span`; the OpenAI Agents SDK's own span context manager has the same blind spot; AutoGen shows it once its orphaned-tool defect is fixed (§6).

the tool-level trace says nothing. Missing executed calls occur in 618 cases, wrong statuses other than false success in 375, and false success in 106. A trace validator that only counts spans would therefore report most incomplete traces as healthy. No case in the corpus fails the phantom, duplicate or isolation checks; they are exercised by the injected defects (Table 1), which fail them.

5.3 Failure modes

Cancellation is reported as success. OpenTelemetry's `use_span` records only `Exception`, and `asyncio.CancelledError` is a `BaseException`. A framework that wraps tool execution in `except Exception` therefore closes a cancelled tool's span with an unset status, which back ends display as success (Figure 5). We observed this in the OpenAI Agents SDK, whose own span context manager ignores the exception type, and in Semantic Kernel and Google ADK; ADK is also affected when it cancels the siblings of a failed parallel call, and AutoGen shows it once its orphaned-tool defect is fixed. Pydantic AI handles cancellation explicitly. Separately, cancellation does not always stop work: AutoGen, LlamaIndex, smolagents and CrewAI let an in-flight tool finish after the caller has seen the run cancelled.

Project	Defect	Scenarios
OpenAI Agents SDK	cancelled function span has no error	S17
AutoGen	`run()` hangs after a `CancellationToken` cancel during a tool; the in-flight tool keeps running after task cancel; once that is fixed, the cancelled tool span is unset	S17
Semantic Kernel	cancelled function span unset (`invoke`, `invoke_stream`)	S17
Google ADK	cancelled tool span unset, including cancelled parallel siblings	S07, S17
smolagents	step memory loses all tool calls when one fails; duplicate call ids silently dropped (reported, not patched)	S05–S07, S11

Table 5: Defects found by TraceGap.

Failed runs lose completed calls. AutoGen's `TaskResult.messages` exists only when a run completes, though its `run_stream()` events keep the calls. When one call in a parallel batch raises, Google ADK's events and Pydantic AI's history drop the results of siblings that succeeded, which is why ADK's events score 77.7% on Stable cases but 14.3% on Stress cases while its spans score 99.2% on Stress. smolagents' memory drops every call of a step in which any call failed.

Rejected calls and nesting. A call to an unknown tool, or with invalid arguments, is often visible only in raw model output. Calls made by an agent used as a tool are missing from the per-agent histories of Pydantic AI and AutoGen, but present in their tracing and OpenTelemetry surfaces.

Fidelity. Across Stable cases 2,980 of 3,090 (96.4%) are faithful. Arguments were recorded exactly by every surface that records them. Error details are the main gap: the OpenAI Agents SDK redacts tool errors from spans under its default error handler, deliberately and for privacy, and Google ADK's events and Pydantic AI's history keep no error when a tool raises; ADK's events also omit some outputs.

6 Defects and fixes

For each defect in Table 5 we wrote a minimal fix against the project's main branch, with a regression test that fails before the fix and passes after; the surrounding test suites show no new failures, and the affected TraceGap scenarios pass on the patched framework. Table 6 lists what is released for each finding.

Project	Finding	Released with the benchmark
OpenAI Agents SDK	cancelled function span has no error	patch, regression test, standalone reproduction
AutoGen	cancellation hang, orphaned tool, unset span	patch, regression tests, standalone reproductions
Semantic Kernel	cancelled function span unset	patch, regression test
Google ADK	cancelled tool spans unset	patch, regression test
smolagents	failed step loses calls; duplicate ids dropped	patch and regression test (lost calls); scenario S11 (duplicate ids)

Table 6: Fixes and reproductions released for each finding. Every finding is also reproduced by the TraceGap scenarios listed in Table 5.

7 History across releases (RQ4)

Because the models are scripted, the same suite can be run against past releases. We ran the core suite against 69 releases published between 2025-03-29 and 2026-10-03, installing each with dependencies resolved as of one day after its release (`uv -exclude-newer`), so each run sees what users installed at the time. Four old Semantic Kernel releases needed current resolution because their original dependency set no longer installs. Of the 69 releases, 65 ran; one had a dependency mismatch, one used an older API, and two

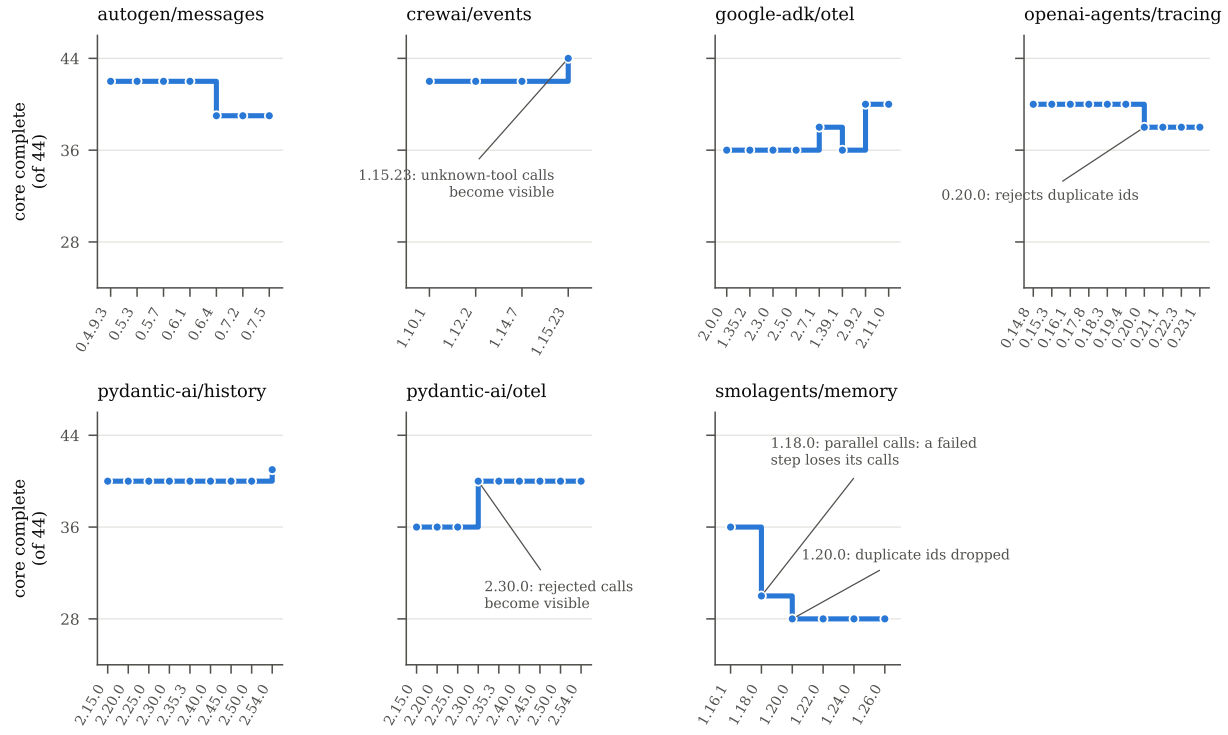


Figure 6: Core-suite completeness (of 44 cases) across releases, for the seven surfaces whose completeness changed. Releases are ordered by date; Google ADK maintained 1.x and 2.x lines in parallel. Annotated changes are confirmed in the per-version verdict diffs. Some changes mix improvements and regressions (smolagents 1.18.0 fixed S03 and S11 while breaking five scenarios), and two (AutoGen 0.6.4, Pydantic AI history 2.54.0) reflect the framework executing a scenario differently rather than a tracing change.

CrewAI releases supported only ReAct-style tool calling. Version–surface pairs that fail the calibration case are left out (11 pairs). Figure 6 shows every surface whose core completeness changed, and Appendix D summarizes all surfaces. The history shows gaps arriving and leaving: smolagents' memory loss arrived together with parallel tool calls in 1.18.0 and its silent dropping of duplicate call ids in 1.20.0; the OpenAI Agents SDK began rejecting duplicate call ids in 0.20.0 and redacting tool errors from spans in 0.23; Pydantic AI's OpenTelemetry spans began showing rejected calls in 2.30.0, and Google ADK's spans across 2.7.1 and 2.9.2; CrewAI's events began showing calls to unknown tools in 1.15.23. Where a framework executed a scenario differently between versions (for example, emitting a different number of calls), the report says so rather than attributing the change to tracing. Run on every release, TraceGap works as a regression suite: each of these changes would have been flagged in the release that introduced it.

8 Dataset and learning tasks (RQ5, RQ6)

Corpus. Every case is released with its three aligned logs (emissions, ledger, recorded trace), labels, tier, the reason for each failure and per-case counters: 6,660 cases for current releases (core and generated) and 4,884 for the 65 past releases, with a calibration flag. The corpus is documented with a datasheet (Gebru et al., 2021) and can be regenerated in minutes. Table 7 summarizes its composition and Appendix F the distribution of defect labels.

Tasks. Because each recorded trace is paired with what really ran, the corpus can evaluate methods that analyse traces, such as trace validators, observability back ends and LLM-based judges. We define tasks that hide the ledger and give a method what a real pipeline sees: the emitted calls, the recorded trace and the

Property	Count
Cases (current releases / past releases)	11,544 (6,660 / 4,884)
Calibrated cases	11,060
Frameworks / trace surfaces / releases	9 / 15 / 65
Emitted tool calls	58,906
Executed calls (ok / error / cancelled / still running)	52,258 (44,638 / 6,644 / 538 / 438)
Emitted but never executed (rejected or never reached)	6,680
Recorded tool-call events	52,830
Stress-tier cases	3,792
Incomplete cases	3,134
Unfaithful cases	1,056
Calls recorded as successful that failed, were cancelled or never ran	222
Duplicate records	0

Table 7: Composition of the released corpus (current and past releases).

run's outcome. The harness's unique `tag` is removed from the inputs, both as a field and as an argument inside emitted and recorded calls, because production tools have no such argument; calls can be joined only by call id, or by name and the remaining arguments. Tag values that some tools echo in their outputs or error messages are replaced by a placeholder. (The scorer still uses tags; only the learning inputs are masked.)

- **T1 Completeness detection** (binary): is the trace complete?
- **T2 Fidelity detection** (binary): are its arguments, outputs and errors exact?
- **T3 Defect classification** (multi-label over ten types): missing call, invisible attempt, false success, other status mismatch, duplicate, phantom, isolation, lost arguments, lost output, lost error. Labels are derived mechanically from the scorer's failure reasons.
- **T4 Execution reconstruction**: for each emitted call, was it ok, error, cancelled, still running, or never executed?

A fifth task, converting a framework's native trace into the canonical record, is the adapter task and is scored by the benchmark itself. Splits hold out whole frameworks, so methods must generalize to unseen trace conventions: train (LangGraph, Pydantic AI, AutoGen, LlamaIndex, CrewAI; 5,884 examples), validation (OpenAI Agents SDK, Semantic Kernel; 2,344) and test (Google ADK, smolagents; 2,832). Uncalibrated rows and harness errors are excluded.

Baselines. We evaluate six methods. A *majority-class* baseline; a *rule-based* baseline that matches emitted to recorded calls by call id and, failing that, one-to-one by name and arguments, and applies rules an engineer could write without the ledger; *logistic regression* and *gradient-boosted trees* (Pedregosa et al., 2011) on 24 hand-designed, surface-agnostic case-level features (counts of emitted, recorded and unmatched calls, recorded statuses, missing outputs and errors, emitted calls that violate the tool schema, run structure, run outcome and mode) and, for T4, 16 call-level features plus the 24 case-level features of the call's case; two further features, the rates at which a surface captures arguments and call ids, characterize a surface's conventions and are evaluated separately below because they hurt generalization; and two models that read the trace as a token sequence: *TF-IDF n-grams with logistic regression* and a small *transformer encoder* trained from scratch (Appendix G). Inputs come from task fields only; tool schemas are treated as known, as they are to any real pipeline, but tool *names* are not used, since the benchmark's names (such as `fail`) reveal behavior that real names would not. T3 uses one binary output per label. The sequence models address T1–T3; T4 uses the feature models.

Detection is largely learnable (RQ5). Table 8 shows that whether a trace is complete can be predicted well from what a pipeline already has: gradient-boosted trees on simple features reach F1 .947 on T1 for frameworks never seen in training. Fidelity loss (T2) is highly detectable in this corpus, largely through status-related features (see below), and its cross-framework evidence is limited to three framework families.

	T1 completeness		T2 fidelity		T3 defects	
Method (test: Google ADK, smolagents)	F1	Bal. acc.	F1	Bal. acc.	Micro F1	Macro F1
Majority class	.000	.500	.000	.500	.000	.000
Rules (matching heuristics)	.795	.843	.085	.497	.373	.258
Logistic regression, features	.933	.946	.983	.998	.877	.789
Gradient-boosted trees, features	.947	.951	.976	.991	.885	.784
TF-IDF + logistic regression	.839	.877	.870	.979	.817	.705
Transformer encoder (from scratch)	.911±.018	.932±.012	.861±.046	.977±.009	.750±.039	.648±.034

Table 8: RQ5: detection without the ledger, on the held-out test frameworks. F1 treats the defect (incomplete, unfaithful) as the positive class. Sequence models: mean ± standard deviation over three training seeds (TF-IDF is deterministic).

	T1 F1			T2 F1			T3 macro F1		
Method	Random	Held-out	LOFO	Random	Held-out	LOFO	Random	Held-out	LOFO
LR, features	.907±.005	.933	.757±.284	.581±.011	.983	.458±.324	.700±.010	.789	.657±.259
GBT, features	.974±.003	.947	.923±.083	.838±.010	.976	.622±.441	.900±.011	.784	.826±.112
TF-IDF + LR	.830±.003	.839	.677±.251	.856±.006	.870	.523±.385	.729±.008	.705	.604±.129
Transformer	.933±.013	.911±.018	.764±.286	.779±.089	.861±.046	.553±.405	.672±.019	.648±.034	.613±.195

Table 9: RQ6: the same methods under three evaluation protocols. Random: 80/20 split of all cases, every framework seen in training (mean ± s.d. over five split seeds for feature models, three for sequence models). Held-out: the official split. LOFO: leave one framework out, mean ± population s.d. across held-out frameworks that have positive cases for the task (nine for T1 and T3, three for T2); the transformer is trained once per fold.

The rule-based baseline, which encodes the natural assumption that an absent call never ran, does markedly worse, and on T4 it is below the majority class in call accuracy (Table 12) because smolagents' memory drops whole steps that did execute and Google ADK's events drop the results of calls that completed. Characterizing *what* was lost is harder: macro F1 on T3 stays well below detection accuracy for every method. The sequence models, which must discover the relevant structure themselves, trail the hand-designed features; the transformer reaches T1 F1 .911 and T3 macro F1 .648.

Random splits overestimate generalization (RQ6). Table 9 evaluates every learned method under three protocols. For all four methods and all three tasks, the leave-one-framework-out average is below the random-split score, by .042 to .332, and it varies widely across held-out frameworks (standard deviation up to .441). For gradient-boosted trees, T3 macro F1 is .900 on a random split but .784 on the official held-out split and .826 averaged over LOFO folds; T4 case exact match falls from .910 to .701 (Table 12). The official test split is itself easier than the LOFO average on T1 and T2 for every method, and on T2 it scores even higher than the random split; a single held-out split can therefore also be optimistic, and LOFO with its spread is the more reliable protocol. Per-framework results (Table 13) show why: frameworks fail in different ways (smolagents drops steps, Google ADK drops sibling results, the OpenAI Agents SDK withholds errors), and a detector learns the failure modes it has seen. A random split, which places cases from the same framework, version and scenario on both sides, therefore overstates how well a trace validator will work on a new framework. We recommend that work on this corpus report LOFO results with their spread. Inferring *how* an unseen framework fails from what its trace omits is the open problem the tasks pose.

T2 is preliminary across frameworks. Fidelity failures occur in only three framework families (Google ADK, Pydantic AI and the OpenAI Agents SDK), so T2's LOFO average rests on three folds, and its large spread (up to ±.441) reflects limited support rather than instability of the method. We treat cross-framework conclusions about T2 as preliminary.

		GBT, held-out split			GBT, LOFO (mean ± s.d.)			LR, held-out	
Features (cumulative)	#	T1 F1	T2 F1	T3 macro	T1 F1	T2 F1	T3 macro	T1 F1	T3 macro
Counts and structure	11	.829	.011	.258	.725±.383	.004±.005	.496±.303	.812	.421
+ recorded statuses	15	.931	.966	.683	.816±.301	.599±.427	.620±.266	.933	.737
+ schema validity	18	.938	.966	.779	.941±.071	.596±.425	.767±.144	.934	.780
+ run outcome and mode	22	.942	.975	.784	.924±.079	.624±.442	.808±.127	.933	.785
+ missing outputs and errors	24	.947	.976	.783	.923±.083	.622±.441	.826±.112	.933	.789
+ captured-field rates (all)	26	.943	.975	.734	.870±.140	.613±.434	.718±.211	.933	.773

Table 10: Feature ablation. The second-to-last row is the surface-agnostic model used as the reference feature baseline in Tables 8–13; the last row adds the two captured-field rates that characterize a surface's conventions.

	Held-out split				LOFO mean	
Method	T1 F1	T2 F1	T3 macro F1	T4 case exact	T1 F1	T3 macro F1
Rules	.795 → .795	.085 → .085	.258 → .258	.601 → .601	.744 → .744	.360 → .360
LR, features	.933 → .933	.983 → .983	.774 → .773	.667 → .667	.762 → .762	.657 → .655
GBT, features	.943 → .943	.975 → .975	.728 → .730	.689 → .698	.870 → .870	.719 → .718

Table 11: Effect of masking the harness tag in task inputs: unmasked → masked.

Are the features shortcuts? Two threats deserve direct tests: that hand-designed features encode the labels almost by definition, and that they let a model recognize the surface and recall its typical failures. Table 10 adds feature groups cumulatively, starting from counts and ending with all 26 features; the reference baselines elsewhere use the surface-agnostic set (second-to-last row). Counts and structure alone already detect incompleteness on the held-out frameworks (T1 F1 .829). T2, in contrast, is close to definitional in this corpus: recorded statuses lift held-out T2 F1 from near zero to above .96, because most fidelity failures are error messages lost when a tool raises, which shows up as error-status records; T2 therefore measures little beyond that here. The features that most directly characterize a surface, the rates at which it captures arguments and call ids, *hurt* generalization: without them, gradient-boosted trees reach LOFO T1 F1 .923 and T3 macro F1 .826, against .870 and .718 with them. On the two held-out test frameworks the same change is smaller (T1 F1 .943 to .947, T3 macro F1 .734 to .783), so a single held-out split would understate it. A probe confirms that the features carry surface identity: a classifier names the surface (one of 15) with 45.1% accuracy from all features, 32.9% without the rates and 13.3% from counts alone, against 8.0% for the majority class (scenario-grouped five-fold cross-validation). A detector that can recognize the surface can fall back on that surface's typical failure modes, which does not transfer to new frameworks. This is a caution for trace validators generally: features that identify the trace surface can cost little on one held-out split while hurting generalization across frameworks, which leave-one-framework-out evaluation exposes. We therefore use the surface-agnostic set for the reference feature baselines throughout.

Masking the harness tag. All task results above use inputs from which the harness's tag has been removed. Table 11 compares them with unmasked inputs, using all 26 features on both sides. Masking changes the feature models' scores by at most .009 on the official split and .012 in mean LOFO score, so the tag did not drive the results: matching by call id and, for calls without a usable id, one-to-one by name and the remaining arguments recovers the same information. The sequence models compare calls only through call ids and never read argument values; their inputs are identical with and without the tag (checked for all 11,060 examples).

9 Discussion and limitations

Several gaps are deliberate design choices: redaction protects sensitive data, and rejecting malformed responses is reasonable. TraceGap does not judge these choices; it makes their consequences visible to people who build on the trace. The cancellation pattern is different: it follows from an interaction between Open-

Method	Call accuracy	Macro F1	Case exact
Majority class	.764	.173	.383
Rules	.741	.404	.601
Logistic regression, features	.823	.620	.651
Gradient-boosted trees, features	.893	.744	.701
GBT, random split (frameworks seen)	.977±.001	.902±.008	.910±.004

Table 12: T4 execution reconstruction on the held-out test frameworks; the last row is the random-split reference (frameworks seen).

		T1 F1					T3 macro F1				
Held-out framework	Examples	Rules	LR	GBT	TF-IDF	Transf.	Rules	LR	GBT	TF-IDF	Transf.
`autogen`	1,328	.915	.656	.953	.628	.644	.431	.909	.956	.722	.583
`crewai`	620	.000	.000	.800	.000	.000	.000	.040	.800	.545	.258
`google-adk`	1,592	.682	.782	.788	.795	.850	.376	.779	.808	.570	.711
`langgraph`	664	.876	.905	.976	.844	.972	.661	.759	.890	.745	.997
`llamaindex`	1,592	.874	.909	.994	.726	.942	.382	.810	.985	.539	.638
`openai-agents`	884	.823	.823	.975	.681	.825	.333	.477	.641	.432	.513
`pydantic-ai`	1,680	.740	.794	.836	.749	.791	.294	.570	.664	.614	.544
`semantic-kernel`	1,460	.939	.952	.994	.791	.920	.255	.650	.889	.442	.492
`smolagents`	1,240	.844	.992	.991	.884	.936	.507	.922	.801	.830	.778

Table 13: Leave-one-framework-out: train on eight frameworks, test on the ninth (all splits pooled; one training run per fold). CrewAI has six defective cases, all from past releases, so its F1 is near zero for most methods: with almost no positives, a few missed cases dominate the score. Per-framework F1 is therefore not a ranking of frameworks or of how hard they are.

Telemetry's exception handling and Python's exception hierarchy that framework authors can easily miss. Whether a cancelled call should be recorded as an *error* is debatable, and some maintainers deliberately keep run cancellation from marking spans; TraceGap requires only that a call that never finished does not read as a success. We suggest that the GenAI semantic conventions state how cancelled tool calls should be recorded.

For machine-learning practice the implication is direct. Trajectories exported from framework traces for evaluation or training, and trace-based judges, inherit these gaps: an evaluator reading a trace that omits rejected attempts or reports cancelled calls as successful will misjudge the agent. Our ablation suggests that span counting, the cheapest validator, misses most such traces, while the learned baselines show that incompleteness itself is detectable from what a pipeline already has.

Limitations. TraceGap covers Python frameworks only. Scripted models exercise framework logic, not provider clients, so defects specific to one provider integration will not appear, and the distribution of conditions differs from natural traffic; the benchmark measures behavior under specified conditions rather than incident rates in production. Attempt visibility is judged at the tool layer; a call visible only in raw model output does not count. CrewAI's events are attributed to runs by task id, information the application has, so its isolation checks are lenient. Cancellation is applied uniformly with `asyncio.Task.cancel()`. Adapters are written by us; the calibration case, the reference loop and mutation testing guard against adapter errors but cannot rule them out, and every failure we report was confirmed by hand. Results apply to the pinned versions. The Stress rule was tested on one run of the released suite made after it was fixed; the two additionally seeded suites were run once each and support the ranking agreement, not the rule. A framework with races of a different structure would need the rule extended. Our learned baselines are deliberately simple; the transformer is trained from scratch because our environment had no access to pretrained weights, and pretrained or LLM-based trace readers are the obvious next step.

Broader impact. TraceGap aims to make agent systems more accountable by checking the records that oversight relies on. Its data is synthetic and contains no personal information. Publishing framework defects could in principle inform attackers who wish to hide actions, but the defects concern honest recording and were reported upstream with fixes; records that do not depend on the audited component, as in TraceGap's ledger, are the general defense.

10 Conclusion

The trace an agent framework produces is infrastructure that other systems trust. TraceGap shows that this trust is mostly but not fully justified, that the gaps concentrate where an accurate record matters most, and that they can be measured reproducibly, ranked with known uncertainty, and fixed. The code, scenarios, results, fixes, corpus and tasks are released at `https://anonymous.4open.science/r/tracegap`.

Reproducibility statement. Every number in this paper is produced by scripts in the released repository from released result files; framework versions are pinned per environment; no API keys or network access are needed to rerun the benchmark. The generated suites are identified by (n, seed).

Use of AI assistance. The benchmark software and an initial draft of this paper were developed with substantial assistance from an AI assistant (Claude, Anthropic). The authors directed the work, reviewed and verified all results, and take full responsibility for the content.

References

Adam AlSayyad, Kelvin Yuxiang Huang, and Richik Pal. AgentTrace: A structured logging framework for agent system observability. *arXiv preprint arXiv:2602.10133*, 2026.

Arize AI. OpenInference specification. `https://github.com/Arize-ai/openinference`, 2026.

Mert Cemri et al. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. TRAIL: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*, 2025.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall, 1993.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

Yasser Hmimou, Mohamed Tabaa, Azeddine Khiat, and Zineb Hidila. A dataset capturing decision processes, tool interactions and provenance links in autonomous AI agents. *Data*, 11(4):66, 2026.

Maurice G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1/2):81–93, 1938.

Xiao Liu et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

OpenTelemetry Authors. Semantic conventions for generative AI systems. `https://opentelemetry.io/docs/specs/semconv/gen-ai/`, 2026.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Jeremy Qin, David Schmotz, Derck Prinzhorn, Luca Beurer-Kellner, Ameya Prabhu, and Maksym Andriushchenko. LLM agents can easily tamper with their own traces. *arXiv preprint arXiv:2609.30266*, 2026.

Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024.

Zahra Seyedghorban, Egor Klimov, Arie van Deursen, Annibale Panichella, and Burcu Kulahcioglu Ozkan. Observability and fault injection for LLM-based multi-agent systems in software engineering. *arXiv preprint arXiv:2608.24271*, 2026.

Yiqi Wang, Jiaqi Zhang, Taotao Cai, Zirui Liu, Qingqiang Sun, Zequn Sun, Zhangkai Wu, Mingkai Zhang, and Yanming Zhu. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. *arXiv preprint arXiv:2606.04990*, 2026.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Shaokun Zhang et al. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *International Conference on Machine Learning*, 2025.

Xinxue Zhu, Jiacong Wu, Xiaoyu Zhang, Tianlin Li, Yanzhou Mu, Juan Zhai, Chao Shen, Chunrong Fang, and Yang Liu. Bugs in modern LLM agent frameworks: An empirical study. *arXiv preprint arXiv:2602.21806*, 2026.

A Core scenarios

Id	Category	Title	Description
S01	basic	Single call	One successful tool call, then a final answer.
S02	basic	Sequential calls	Three tool calls across three model turns.
S03	basic	Parallel calls	Five tool calls emitted in one model turn.
S04	basic	Long session	Twenty-five sequential tool turns (limits raised so the run can finish).
S05	failure	Tool raises	A tool raises an exception. The trace should record the call as failed.
S06	failure	Failure then retry	A failing call is followed by a successful call in the next turn.
S07	failure	Parallel batch with one failure	Three calls in one turn; the middle one raises. Siblings may or may not be cancelled.
S08	rejected	Unknown tool	The model calls a tool that does not exist. Nothing executes; the attempt should still be visible.
S09	rejected	Invalid argument type	The model passes a string where an integer is required.
S10	rejected	Missing required argument	The model omits a required argument.
S11	identity	Duplicate call ids	Two different calls in one turn share the same call id (a real provider glitch).
S12	payload	Unicode and escape characters	Arguments contain emoji, quotes, newlines, backslashes and non-Latin script.
S13	payload	Large output	A tool returns 200,000 characters.
S14	payload	Structured output	A tool returns a dict rather than a string.
S15	payload	None output	A tool returns None.
S16	payload	Synchronous tool	A plain (non-async) tool, which frameworks typically run in a worker thread.
S17	interrupt	Cancelled during a tool	The run is cancelled 0.3 s into a 2 s tool call. The call started but never finished.
S18	interrupt	Cancelled after tools completed	Two tools finish, then the run is cancelled while the model is thinking. Completed calls must not disappear.
S19	interrupt	Model error after tools	Two tools finish, then the model provider fails. Completed calls must not disappear.
S20	interrupt	Turn limit exceeded	The model keeps calling tools past the framework's turn limit (3 model calls).
S21	structure	Concurrent runs	Two runs of the same agent execute concurrently. Each run's trace must contain only its own calls.
S22	structure	Nested agent as tool	The outer agent calls a 'delegate' tool that runs an inner agent, which calls its own tools.

B Trace surfaces

Surface	What is read
`langgraph/callbacks`	LangChain `on_tool_start/end/error` callbacks
`openai-agents/tracing`	`FunctionSpanData` spans via a `TracingProcessor`
`pydantic-ai/history`	`capture_run_messages()` / `run_stream_events()`
`pydantic-ai/otel`	`execute_tool` OpenTelemetry spans
`autogen/messages`	`ToolCallRequest/ExecutionEvent` from `run()` / `run_stream()`
`autogen/otel`	autogen-core `execute_tool` spans (name and call id only)
`smolagents/memory`	`agent.memory.steps` (presence, arguments and call ids only)
`smolagents/openinference`	OpenInference `TOOL` spans
`llamaindex/events`	`ToolCall` / `ToolCallResult` workflow events
`llamaindex/openinference`	OpenInference `TOOL` spans
`semantic-kernel/filters`	a `FUNCTION_INVOCATION` filter
`semantic-kernel/otel`	Semantic Kernel GenAI diagnostics spans
`google-adk/events`	`Runner.run_async` events
`google-adk/otel`	ADK `execute_tool` spans
`crewai/events`	`ToolUsageStarted/Finished/Error` on the CrewAI event bus

C Ranking robustness across generated suites

Stable-tier completeness (%) per surface on independently generated suites: seed 1 (n=200, the released suite) and seeds 2, 3 (n=100).

Surface	Seed 1	Seed 2	Seed 3
`crewai/events`	100.0	100.0	100.0
`google-adk/otel`	91.5	93.3	78.0
`llamaindex/events`	90.2	82.2	92.7
`pydantic-ai/history`	89.0	91.1	91.5
`pydantic-ai/otel`	78.0	71.1	73.2
`autogen/otel`	74.4	62.2	65.9
`langgraph/callbacks`	74.4	60.0	75.6
`google-adk/events`	72.0	86.7	85.4
`openai-agents/tracing`	74.4	62.2	65.9
`autogen/messages`	70.7	72.2	70.7
`llamaindex/openinference`	65.9	51.1	58.5
`semantic-kernel/filters`	61.0	55.6	63.4
`semantic-kernel/otel`	53.7	48.9	48.8
`smolagents/openinference`	36.6	35.6	43.9
`smolagents/memory`	31.7	35.6	34.1

Individual rates move by up to 16 points between suites (largest for `langgraph/callbacks`), consistent with the bootstrap intervals in Figure 2. `crewai/events` ranks first in every suite and the same three surfaces (`semantic-kernel/otel`, `smolagents/memory`, `smolagents/openinference`) occupy the last three places in every suite.

D History per surface

Surface	Versions	Release dates	Core complete (of 44)
`autogen/messages`	7	2025-03-29 – 2025-09-30	39–42
`autogen/otel`	3	2025-07-09 – 2025-09-30	40–40
`crewai/events`	4	2026-03-04 – 2026-09-28	42–44
`google-adk/events`	8	2026-05-19 – 2026-10-02	44–44
`google-adk/otel`	8	2026-05-19 – 2026-10-02	36–40
`langgraph/callbacks`	5	2025-07-21 – 2026-09-21	40–40
`llamaindex/events`	8	2025-07-22 – 2026-09-21	42–42
`llamaindex/openinference`	8	2025-07-22 – 2026-09-21	38–38
`openai-agents/tracing`	10	2026-04-29 – 2026-10-02	38–40
`pydantic-ai/history`	9	2026-07-22 – 2026-10-03	40–41
`pydantic-ai/otel`	9	2026-07-22 – 2026-10-03	36–40
`semantic-kernel/filters`	8	2025-05-15 – 2026-08-06	40–40
`semantic-kernel/otel`	5	2025-09-04 – 2026-08-06	38–38
`smolagents/memory`	6	2025-05-16 – 2026-05-29	28–36
`smolagents/openinference`	2	2026-01-16 – 2026-05-29	34–34

Uncalibrated version–surface pairs are excluded. Per-version results and every verdict change are in the released history report.

E Cost

Surface	Median seconds per case	Total minutes (both suites)
`autogen/messages`	0.075	4.0
`autogen/otel`	0.078	4.0
`crewai/events`	0.126	6.3
`google-adk/events`	0.075	1.0
`google-adk/otel`	0.077	1.0
`langgraph/callbacks`	0.077	1.4
`llamaindex/events`	0.107	4.2
`llamaindex/openinference`	0.126	4.3
`openai-agents/tracing`	0.076	1.9
`pydantic-ai/history`	0.064	1.2
`pydantic-ai/otel`	0.074	1.3
`semantic-kernel/filters`	0.067	2.3
`semantic-kernel/otel`	0.067	2.3
`smolagents/memory`	0.080	5.8
`smolagents/openinference`	0.091	5.9

Times are wall clock on a two-core cloud machine, dominated by deliberate delays in timing scenarios.

F Defect labels

Distribution of T3 defect labels over calibrated cases of both splits (a case can carry several labels). Labels with zero count occur only under injected defects (Table 1).

Label	Cases
`missing_call`	745
`invisible_attempt`	1,890
`false_success`	200
`status_mismatch`	391
`duplicate`	0
`phantom`	0
`isolation`	0
`args_lost`	0
`output_lost`	376
`error_lost`	856

G Sequence models

Serialization. Each example becomes a token sequence (at most 256 tokens): a mode token; run-outcome tokens; then, for each emitted call, its position, schema validity, run index and nesting; then, for each recorded event, the position of the emitted call it refers to by call id (or an unknown marker), its status, whether output, error, arguments and call id are present, and a group index. Tool names and argument values are not used.

TF-IDF + logistic regression. 1–3-gram TF-IDF over the token sequence (minimum document frequency 2, sublinear term frequency), class-balanced logistic regression with C=4, one model per binary label.

Transformer. A two-layer encoder with four heads, width 64, learned positional embeddings and a CLS token, trained from scratch in JAX with AdamW (learning rate 2×10^{-3}, cosine schedule with warm-up, weight decay 0.01, batch 64, 6 epochs) and a shared trunk with heads for T1, T2 and the ten T3 labels (class-weighted binary cross-entropy). Our compute environment had no access to pretrained model weights, so we train from scratch; pretrained encoders are a natural next baseline.

H Task features

Case features (24 in the reference baselines, 26 in the ablation): numbers of emitted and recorded calls and their difference; share of emitted calls matched; unmatched emitted calls; emitted calls matched more than once; unmatched records; counts of recorded ok, error, incomplete and unknown statuses; emitted calls violating the tool schema, in total and among unmatched calls; unmatched schema-valid calls; ok records without output; error records without a message; shares of records without arguments and without call ids (the two surface-characterizing features, used only in the ablation); number of groups, number of runs and whether groups exceed runs; nested emitted calls; mode; and run-outcome indicators (completed, cancelled, raised). Call features (16, combined with the case features of the call's case for T4): whether and how often the call is matched; the matched record's status indicators; whether its output is missing and its error present; schema validity; nesting; relative position; whether the call comes after the last matched call of its run; whether it is last; and the run-outcome indicators.